1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality.

Aim

To minimize passenger waiting time and improve ride-sharing service using Reinforcement Learning.

Agent

Ride-sharing dispatch system.

Environment

Drivers, passengers, and city traffic.

State Space (S)

- Driver location
- Passenger location
- Traffic condition
- Waiting time
- Driver availability

Action Space (A)

- Assign Driver 1
- Assign Driver 2
- Wait

Reward Function (R)

- Quick pickup: **+10**
- Ride completed: **+20**
- Long waiting time: **−10**
- Ride cancelled: **−20**

Simple Python Code

```
import numpy as np

import random

# States (Passenger Locations)

states = ["North", "South", "East", "West"]

# Actions (Drivers)

actions = ["Driver_A", "Driver_B", "Driver_C"]
```

```
# Q-Table
Q = np.zeros((len(states), len(actions)))
# Parameters
alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 1000
# Reward Function
def get_reward(state, action):
    if state == 0 and action == 0:
        return 10
    elif state == 1 and action == 1:
        return 10
    elif state == 2 and action == 2:
        return 10
    elif state == 3 and action == 0:
        return 8
    else:
        return -2
# Training
for episode in range(episodes):
state = random.randint(0, len(states) - 1)
    if random.uniform(0, 1) < epsilon:
        action = random.randint(0, len(actions) - 1)
    else:
        action = np.argmax(Q[state])
```

```
reward = get_reward(state, action)

next_state = random.randint(0, len(states) - 1)

Q[state][action] = Q[state][action] + alpha * (

      reward + gamma * np.max(Q[next_state]) - Q[state][action]

   )

# Display Q-Table

print("Learned Q-Table\n")

print(Q)

# Best Driver for Each Passenger Location

print("\nOptimal Driver Assignment\n")

for i in range(len(states)):

   best_action = np.argmax(Q[i])

   print("Passenger at", states[i], "-> Assign", actions[best_action])
```

Output

```
========== RESTART: C:/Users/SAMSUNG/OneDrive/I
Learned Q-Table

[[85.29847253 40.82879762 46.75845534]
 [46.17121921 84.40651411 53.34277107]
 [50.64699514 28.31772107 84.91909742]
 [82.84220456 44.52798278 53.77544392]]

Optimal Driver Assignment

Passenger at North -> Assign Driver_A
Passenger at South -> Assign Driver_B
Passenger at East -> Assign Driver_C
Passenger at West -> Assign Driver_A
>>>
```

2. An e-learning platform uses Reinforcement Learning to recommend personalizedstudy materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations.

Aim

To recommend personalized study materials using Reinforcement Learning.

Agent

Recommendation system.

Environment

Students and the e-learning platform.

State

- Student's progress
- Previous quiz scores
- Learning history

Actions

- Recommend familiar content
- Recommend new content

Exploration vs Exploitation

- **Exploration:** Recommends new study materials to discover better learning resources.
- **Exploitation:** Recommends study materials that have previously helped the student perform well.

Reward

- High quiz score: **+10**
- Course completed: **+20**
- Low score: **−5**

Python Code

```
import random

# Study materials

materials = ["Python", "Machine Learning", "Data Science", "NLP"]

# Initial rewards

values = [0, 0, 0, 0]

counts = [0, 0, 0, 0]

epsilon = 0.2

rounds = 20

for i in range(rounds):

 Exploration or Exploitation

    if random.random() < epsilon:

        choice = random.randint(0, len(materials) – 1
```

```
strategy = "Explore"
    else:
        choice = values.index(max(values))
        strategy = "Exploit"
 Simulated student feedback (reward)
    reward = random.randint(1, 10)
 counts[choice] += 1
    values[choice] += (reward - values[choice]) / counts[choice]
print(strategy, "->", materials[choice], "Reward:", reward)
print("\nAverage Reward of Each Material")
for i in range(len(materials)):
print(materials[i], ":", round(values[i], 2))
best = values.index(max(values))print("\nRecommended Study Material:", materials[best])
```

Output

```
Exploit -> Python Reward: 6
Exploit -> Python Reward: 9
Exploit -> Python Reward: 4
Exploit -> Python Reward: 8
Exploit -> Python Reward: 7
Explore -> Data Science Reward: 9
Exploit -> Data Science Reward: 7
Explore -> NLP Reward: 8
Exploit -> Data Science Reward: 2
Explore -> Data Science Reward: 1
Exploit -> NLP Reward: 7
Exploit -> NLP Reward: 3
Exploit -> Python Reward: 8
Exploit -> Python Reward: 7
Exploit -> Python Reward: 6
Explore -> NLP Reward: 10
Exploit -> NLP Reward: 8
Exploit -> NLP Reward: 7
Explore -> Machine Learning Reward: 7
Exploit -> NLP Reward: 6

Average Reward of Each Material
Python : 6.88
Machine Learning : 7.0
Data Science : 4.75
NLP : 7.0

Recommended Study Material: Machine Learning
```

Result

Thus, Reinforcement Learning balances **exploration** (trying new study materials) and **exploitation** (recommending familiar, effective content) to provide personalized recommendations and improve student learning performance.

3. Design and develop a Reinforcement Learning framework a robotic vacuum

cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles.

Apply the concept of policy and explain how the value function improves performance.

over time.

Aim

To clean a room efficiently while avoiding obstacles using Reinforcement Learning.

Agent

Robotic vacuum cleaner.

Environment

Room with dirt, furniture, and obstacles.

State

- Robot position
- Dirt location
- Obstacle location
- Battery level

Actions

- Move Forward
- Move Left
- Move Right
- Clean
- Stop

Policy

A **policy** is a set of rules that decides the best action for each state. For example, if dirt is detected ahead, the robot chooses **Clean**; if an obstacle is detected, it turns left or right.

Value Function

The **value function** estimates how good a state is based on the expected future rewards. Over time, the robot learns which paths clean more dirt with fewer obstacles and less battery usage, improving its performance.

Reward

- Clean dirt: **+10**
- Reach clean area: **+5**
- Hit obstacle: **−10**
- Waste battery: **−2**

Python Code

```
import random

 States (Rooms)

rooms = ["Room1", "Room2", "Room3", "Room4"]

# Actions

actions = ["Clean", "Move"]

# Value Function

values = [0, 0, 0, 0]

alpha = 0.1
```

```
gamma = 0.9
episodes = 20
for episode in range(episodes):
state = random.randint(0, 3)
    action = random.choice(actions)
# Reward
    if action == "Clean":
        reward = 10
    else:
        reward = -2
# Value Function Update
    values[state] = values[state] + alpha * (
        reward + gamma * max(values) - values[state]
    )
  print("State:", rooms[state],
          "| Action:", action,
          "| Reward:", reward)
print("\nState Values")
for i in range(4):
    print(rooms[i], ":", round(values[i], 2))
best = values.index(max(values))
print("\nBest Room to Clean Next:", rooms[best])
```

Output

```
State: Room2 | Action: Move | Reward: -2
State: Room3 | Action: Clean | Reward: 10
State: Room2 | Action: Move | Reward: -2
State: Room2 | Action: Move | Reward: -2
State: Room2 | Action: Move | Reward: -2
State: Room4 | Action: Move | Reward: -2
State: Room4 | Action: Clean | Reward: 10
State: Room4 | Action: Move | Reward: -2
State: Room4 | Action: Move | Reward: -2
State: Room4 | Action: Clean | Reward: 10
State: Room3 | Action: Clean | Reward: 10
State: Room2 | Action: Clean | Reward: 10
State: Room1 | Action: Clean | Reward: 10
State: Room1 | Action: Move | Reward: -2
State: Room4 | Action: Clean | Reward: 10
State: Room2 | Action: Clean | Reward: 10
State: Room3 | Action: Move | Reward: -2
State: Room3 | Action: Clean | Reward: 10
State: Room2 | Action: Move | Reward: -2
State: Room4 | Action: Move | Reward: -2

State Values
Room1 : 1.05
Room2 : 1.81
Room3 : 2.93
Room4 : 2.45

Best Room to Clean Next: Room3
```

Result

Thus, the Reinforcement Learning framework enables the robotic vacuum cleaner to learn the best cleaning policy using the value function, resulting in efficient cleaning, obstacle avoidance, and improved performance over time.

4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance.

Aim

To maximize profit by deciding whether to **Buy, Sell, or Hold** using Reinforcement Learning.

Agent

Cryptocurrency trading system.

Environment

Cryptocurrency market.

State

- Current price
- Market trend
- Previous price
- Portfolio balance

Actions

- Buy
- Sell
- Hold

Challenges in Reward Function

- Market prices change rapidly.
- Balancing short-term and long-term profit.
- Avoiding excessive trading.
- Managing financial risk.

Effect of Poor Reward Design

- Frequent unnecessary trades.
- Higher financial losses.
- Poor investment decisions.
- Slow or unstable learning.

Reward

- Profit: **+10**
- Hold during stable market: **+5**
- Loss: **−10**

Python Code

```
import random

# Actions

actions = ["Buy", "Sell", "Hold"]

profit = 0

for i in range(10):

    action = random.choice(actions)

    # Simulated reward

    reward = random.randint(-10, 10)

    profit += reward
```

```
    print("Action:", action, "| Reward:", reward)

print("\nTotal Profit:", profit)

if profit > 0:

    print("Trading Performance: Profit")

else:

    print("Trading Performance: Loss")
```

Output

```
Action: Sell | Reward: 8
Action: Hold | Reward: 5
Action: Hold | Reward: 5
Action: Buy | Reward: 2
Action: Hold | Reward: -2
Action: Hold | Reward: -9
Action: Buy | Reward: -3
Action: Sell | Reward: 9
Action: Sell | Reward: 6
Action: Sell | Reward: -5

Total Profit: 16
Trading Performance: Profit
```

Result

Thus, a well-designed reward function helps the RL agent make profitable trading decisions, while a poor reward function leads to inefficient trading and reduced overall performance.

5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism.

Aim

To manage electricity distribution efficiently using Reinforcement Learning.

Agent

Smart grid controller.

Environment

Power grid with consumers and electricity supply.

State Space

- Electricity demand
- Power supply
- Battery level
- Peak/Off-peak time

Action Space

- Increase power supply
- Decrease power supply
- Store energy
- Distribute stored energy

Reward Mechanism

- Balanced power distribution: **+10**
- Energy saving: **+5**
- Power shortage: **−10**
- Energy wastage: **−5**

Python Code

```
import random

# State Space (Power Demand)

states = ["Low", "Medium", "High"]

# Action Space

actions = ["Increase Power", "Decrease Power", "Maintain Power"]

total_reward = 0

for i in range(10):

    state = random.choice(states)

    action = random.choice(actions)

# Reward Mechanism

    if state == "High" and action == "Increase Power":

        reward = 10

    elif state == "Low" and action == "Decrease Power":

        reward = 8

    elif state == "Medium" and action == "Maintain Power":

        reward = 9
```

```
    else:
        reward = -3
    total_reward += reward
    print("State:", state,
          "| Action:", action,
          "| Reward:", reward)
print("\nTotal Reward:", total_reward)
if total_reward > 0:
    print("Smart Grid Performance: Efficient")
else:
    print("Smart Grid Performance: Inefficient")
```

Output

```
State: Medium | Action: Decrease Power | Reward: -3
State: Low | Action: Decrease Power | Reward: 8
State: High | Action: Decrease Power | Reward: -3
State: Medium | Action: Maintain Power | Reward: 9
State: High | Action: Increase Power | Reward: 10
State: Medium | Action: Increase Power | Reward: -3
State: High | Action: Maintain Power | Reward: -3
State: Low | Action: Maintain Power | Reward: -3
State: Medium | Action: Maintain Power | Reward: 9
State: Low | Action: Increase Power | Reward: -3

Total Reward: 18
Smart Grid Performance: Efficient
```

Result

Thus, the Reinforcement Learning model efficiently manages electricity distribution by defining the **state space, action space, and reward mechanism**, reducing power wastage and improving energy efficiency.

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes.

Aim

To adjust medication dosages and improve patient health using Reinforcement Learning.

Agent

Healthcare monitoring system.

Environment

Patient's health condition.

State

- Heart rate
- Blood pressure
- Blood sugar level
- Patient condition

Actions

- Increase dosage
- Decrease dosage
- Maintain current dosage

Reward

- Improved health: **+10**
- Stable condition: **+5**
- Side effects: **−10**
- Worsening condition: **−20**

Learning Improvement

The RL agent learns from patient responses and gradually selects the most effective dosage, improving treatment outcomes while reducing side effects.

Python Code

```
import random

# States (Patient Condition)
states = ["Low", "Normal", "High"]

# Actions (Medication Dosage)
actions = ["Increase Dose", "Decrease Dose", "Maintain Dose"]

total_reward = 0

for i in range(10):

    state = random.choice(states)
    action = random.choice(actions)

    # Reward Function
```

```
    if state == "Low" and action == "Increase Dose":
        reward = 10
    elif state == "High" and action == "Decrease Dose":
        reward = 10
    elif state == "Normal" and action == "Maintain Dose":
        reward = 10
    else:
        reward = -5

    total_reward += reward

    print("State:", state,
          "| Action:", action,
          "| Reward:", reward)

print("\nTotal Reward:", total_reward)

if total_reward > 0:
    print("Patient Health Improved")
else:
    print("Needs More Training")
```

Sample Output

```
========== RESTART: C:/Users/SAMSUNG/OneDrive/Desktop/RLMLA03/EXP6.PY ==========
State: High | Action: Decrease Dose | Reward: 10
State: Low | Action: Maintain Dose | Reward: -5
State: High | Action: Decrease Dose | Reward: 10
State: Low | Action: Increase Dose | Reward: 10
State: Normal | Action: Decrease Dose | Reward: -5
State: Normal | Action: Increase Dose | Reward: -5
State: Low | Action: Decrease Dose | Reward: -5
State: High | Action: Maintain Dose | Reward: -5
State: Low | Action: Decrease Dose | Reward: -5
State: Low | Action: Increase Dose | Reward: 10

Total Reward: 10
Patient Health Improved
```

Result

Thus, the Reinforcement Learning model learns the optimal medication dosage by using **states, actions, and rewards**, leading to better patient health and improved treatment outcomes.

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization.

Aim

To optimize drone navigation and delivery routes using Reinforcement Learning.

Agent

Delivery drone.

Environment

Delivery area with roads, buildings, obstacles, and weather conditions.

State

- Drone location
- Destination
- Battery level
- Obstacles

Actions

- Move Forward
- Turn Left
- Turn Right
- Deliver Package
- Return to Base

Exploration vs Exploitation

- **Exploration:** Tries new routes to find shorter or safer paths.
- **Exploitation:** Uses the best previously learned route for faster delivery.

Reward

- Successful delivery: **+20**
- Shortest route: **+10**
- Collision or obstacle hit: **−10**
- Battery wasted: **−5**

Python Code

```
import random

# Possible Routes

routes = ["Route A", "Route B", "Route C"]

# Average rewards and visit counts

values = [0, 0, 0]

counts = [0, 0, 0]

epsilon = 0.2

rounds = 10

for i in range(rounds):
```

```
    # Exploration or Exploitation
    if random.random() < epsilon:
        choice = random.randint(0, 2)
        strategy = "Explore"
    else:
        choice = values.index(max(values))
        strategy = "Exploit"
    # Simulated reward
    reward = random.randint(1, 10)
    counts[choice] += 1
    values[choice] += (reward - values[choice]) / counts[choice]
    print(strategy, "|", routes[choice], "| Reward:", reward)
print("\nAverage Reward")
for i in range(3):
    print(routes[i], ":", round(values[i], 2))
best = values.index(max(values))
print("\nBest Route:", routes[best])
```

Output

```
---------- RESTART: C:/Users/SAMSUNG/OneDrive/
Explore | Route A | Reward: 3
Exploit | Route A | Reward: 4
Exploit | Route A | Reward: 9
Exploit | Route A | Reward: 1
Explore | Route C | Reward: 7
Exploit | Route C | Reward: 6
Exploit | Route C | Reward: 5
Exploit | Route C | Reward: 3
Exploit | Route C | Reward: 9
Explore | Route C | Reward: 1

Average Reward
Route A : 4.25
Route B : 0
Route C : 5.17

Best Route: Route C
```

Result

Thus, Reinforcement Learning helps the drone learn optimal delivery routes by balancing **exploration** and **exploitation**, resulting in faster, safer, and more efficient deliveries.

8. Design and develop a Reinforcement Learning framework a manufacturing robot

uses Reinforcement Learning to optimize assembly operations. Apply the concept of

policy and explain how value functions help improve efficiency.

Aim

To optimize assembly operations using Reinforcement Learning.

Agent

Manufacturing robot.

Environment

Assembly line with machines, parts, and workstations.

State

- Robot position
- Part availability
- Machine status
- Task completion

Actions

- Pick part
- Assemble part
- Move to next station
- Wait

Policy

A **policy** decides the best action in each state. For example, if a part is available, the robot selects **Pick Part** and then **Assemble Part**.

Value Function

The **value function** estimates the future reward of each state. Over time, the robot learns the most efficient sequence of actions, reducing assembly time and increasing productivity.

Reward

- Successful assembly: **+10**
- Task completed quickly: **+5**
- Wrong assembly: **−10**
- Delay: **−5**

Python Code

```
import random

# States (Assembly Stages)

states = ["Part Loading", "Assembly", "Quality Check"]

# Actions

actions = ["Pick", "Assemble", "Inspect"]

# Value Function

values = [0, 0, 0]

alpha = 0.1

gamma = 0.9

episodes = 10

for i in range(episodes):

    state = random.randint(0, 2)

    action = random.choice(actions)
```

```
    # Reward Function
    if state == 0 and action == "Pick":
        reward = 10
    elif state == 1 and action == "Assemble":
        reward = 10
    elif state == 2 and action == "Inspect":
reward = 10
    else:
        reward = -5
    # Value Function Update
    values[state] = values[state] + alpha * (
        reward + gamma * max(values) - values[state]
    )
    print("State:", states[state],
          "| Action:", action,
          "| Reward:", reward)
print("\nState Values")
for i in range(3):
    print(states[i], ":", round(values[i], 2))
best = values.index(max(values))
print("\nBest State:", states[best])
```

Output

```
========== RESTART: C:/Users/SAMSUNG/OneDrive/Desktop/RLMLA03
State: Assembly | Action: Inspect | Reward: -5
State: Quality Check | Action: Assemble | Reward: -5
State: Assembly | Action: Pick | Reward: -5
State: Assembly | Action: Assemble | Reward: 10
State: Quality Check | Action: Inspect | Reward: 10
State: Quality Check | Action: Inspect | Reward: 10
State: Part Loading | Action: Pick | Reward: 10
State: Quality Check | Action: Assemble | Reward: -5
State: Part Loading | Action: Assemble | Reward: -5
State: Assembly | Action: Pick | Reward: -5

State Values
Part Loading : 0.63
Assembly : -0.28
Quality Check : 1.04

Best State: Quality Check
```

Result

Thus, the Reinforcement Learning framework enables the manufacturing robot to learn the optimal policy using the value function, improving assembly efficiency, reducing errors, and increasing productivity.

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance.

Aim

To maximize the click-through rate (CTR) using Reinforcement Learning.

Agent

Online advertising system.

Environment

Users browsing websites or mobile applications.

State

- User interests
- Browsing history
- Time of day
- Ad category

Actions

- Show Ad A
- Show Ad B
- Show Ad C

Reward Design Challenges

- Predicting user preferences.
- Balancing clicks and long-term engagement.
- Avoiding repeated ads.
- Handling changing user behavior.

Impact of Poor Reward Design

- Low click-through rate.
- Repeated irrelevant ads.
- Poor user experience.
- Reduced advertising revenue.

Reward

- Ad clicked: **+10**
- Ad ignored: **−5**

Python Code

```
clicked = True

if clicked:
    reward = 10
else:
    reward = -5

print("Reward:", reward)
```

Sample Output

```
---------- RESTART: C:/Users/SAMSUNG/OneDrive/
Advertisement: Ad A | Click: 0
Advertisement: Ad B | Click: 0
Advertisement: Ad B | Click: 0
Advertisement: Ad C | Click: 1
Advertisement: Ad B | Click: 1
Advertisement: Ad A | Click: 1
Advertisement: Ad C | Click: 0
Advertisement: Ad A | Click: 1
Advertisement: Ad A | Click: 0
Advertisement: Ad A | Click: 1

Total Clicks: 5
Performance: Good
```

Result

Thus, an effective reward function helps the RL system display relevant advertisements, increasing click-through rates and improving overall advertising performance.

10. A smart irrigation system uses Reinforcement Learning to optimize water usage.

Create an RL framework by defining states, actions, and reward function.

Aim

To optimize water usage in agriculture using Reinforcement Learning.

Agent

Smart irrigation controller.

Environment

Farm with crops, soil, and weather conditions.

State

- Soil moisture
- Weather condition
- Water level
- Crop growth stage

Actions

- Turn irrigation ON
- Turn irrigation OFF
- Increase water supply
- Reduce water supply

Reward Function

- Proper irrigation: **+10**
- Water saving: **+5**
- Overwatering: **−10**
- Underwatering: **−5**

Python Code

```
import random

# States (Soil Moisture)

states = ["Dry", "Normal", "Wet"]

# Actions

actions = ["Increase Water", "Maintain Water", "Stop Water"]

total_reward = 0

for i in range(10):

    state = random.choice(states)
```

```
action = random.choice(actions)
    # Reward Function
    if state == "Dry" and action == "Increase Water":
        reward = 10
    elif state == "Normal" and action == "Maintain Water":
        reward = 8
    elif state == "Wet" and action == "Stop Water":
        reward = 10
    else:
        reward = -5
    total_reward += reward
    print("State:", state,
          "| Action:", action,
          "| Reward:", reward)
print("\nTotal Reward:", total_reward)
if total_reward > 0:
 print("Irrigation Performance: Efficient")
else:
    print("Irrigation Performance: Inefficient")
```

Output

```
========== RESTART: C:/Users/SAMSUNG/OneDrive/Desktop/RLMLA03/EXP10.PY
State: Wet | Action: Maintain Water | Reward: -5
State: Normal | Action: Maintain Water | Reward: 8
State: Dry | Action: Stop Water | Reward: -5
State: Dry | Action: Maintain Water | Reward: -5
State: Dry | Action: Maintain Water | Reward: -5
State: Wet | Action: Maintain Water | Reward: -5
State: Normal | Action: Increase Water | Reward: -5
State: Wet | Action: Increase Water | Reward: -5
State: Wet | Action: Maintain Water | Reward: -5
State: Wet | Action: Increase Water | Reward: -5

Total Reward: -37
Irrigation Performance: Inefficient
```

Result

Thus, the Reinforcement Learning framework optimizes water usage by defining appropriate **states, actions, and reward functions**, improving crop growth while reducing water wastage.

.